

AI-Assisted Workflow -- Prompt Highlights

Career Fair Eligibility Shortlist | Samyak Sanklecha

A curated trail of how the solution was built with AI: translating the spec, an AI-influenced design decision with trade-offs, a real iteration where I caught and fixed a validation gap, and the test-driven red-to-green progression. Full history available on request.

1. Understanding the spec + choosing the tech stack -- translating the problem statement into a technical brief and requirements. (The highest-weighted 'spec to technical spec' prompt.)

Dev work file access

We will be working on a mini - project now and it based on career fair student shortlist -> You need to go to my downloads and browse through the entire problem statement and load it into your memory (I do not want you to keep fetching the details by reading the document repeatedly) . Get a good understanding of the project .
Once this is done I want you to list out all the functional requirements + non-functional requirements for the project .
I also want to mention that we will be working with a test-driven development approach and frame our test suite first before starting the implementation
Now the main deliverables (We will be doing this one by one these are the phases)-
1)I want you to recommend me a tech stack for this project which should be simple enough to explain the core functionality (Like an HTML page with JS rendered using Vanilla JS and CSS for styling - correct me if im wrong or if there's a better tech stack available)
2)The design phase starts here - we will discuss the feasible high level + low level architecture
3)We will generate the test suite with all test cases (and review them together) and move forward
4)Start with the implementation and further work along with testing

Show less

37 minutes ago

Spec is fully loaded — I read the complete P06 statement earlier this session (profile, the five roles, all the contracts, and every acceptance criterion), so it's already in context. No need to re-open the file, which also keeps us token-efficient. And noted on TDD — test suite before implementation. That's the right call for this problem and we'll do it in Phase 3.

Here's the requirements breakdown, then my Phase 1 tech-stack recommendation.

Functional Requirements

#	Requirement
FR1	Editable student profile: branch, CGPA, graduation year, active backlogs, skills
FR2	Display the five fixed roles (CF01–CF05) with their requirements

+ Write a message...

Add folder Manual Opus 4.8 Medium

okay works having a seperate logic file also works to help us work with TDD along with a seperate Test suite file
Now yes i want confirmation on the tech stack for us to move ahead with .

Confirmed. Locking the stack:

Tech stack: vanilla HTML + CSS + JavaScript — no framework, no build, no dependencies.

File layout (3 files):

- `logic.js` — pure functions only: validation, eligibility evaluation, ordering. No DOM.
- `index.html` — UI + styling, imports `logic.js`.
- `tests.js` — assertion-based test suite against `logic.js`, run with `node tests.js`.

One-line justification for each (for your talk track):

- Vanilla, no framework → matches the scope, avoids the over-engineering the guide penalizes, and you can explain every line.
- No build step → instant refresh, which is what makes the live-modification round painless.
- Pure logic split out from the DOM → the thing that makes TDD actually possible and makes live edits low-risk (change a rule in one place ↓ sts confirm it, UI untouched).

+ Write a message...

Add folder Manual Opus 4.8 Medium

2. AI-influenced design decision: roles as a data-driven config vs. hardcoded -- with the trade-off reasoning (configuration over code).

Dev work file access

Before we lock the design I have two data structure decisions I want to reason through with trade-offs.

1) Should the five roles be hardcoded in `evaluateRole` function, or stored as a pure data-driven config array so adding/removing a role is a data change, not a code change?

2) Should the fixed failure-reason order live implicitly in the sequence of code, or as an explicit ordered list of rule definitions? Give me the trade-offs of each and a recommendation, keeping our 'simple, explainable, easy to modify live' constraints in mind.

Good — both are real forks worth reasoning through. Here's my analysis against our constraints (simple, explainable, easy to modify live).

Decision 1 — Roles: hardcoded vs. data-driven config

Option A — hardcoded (per-role logic, or a switch inside `evaluateRole`):

- Pro: each role's rules are visible right where they're checked.
- Con: duplication, `evaluateRole` bloats, adding a role means *writing and testing new code*, and the logic and the data are tangled.

Option B — data-driven config array: roles are plain data; one generic `evaluateRole(profile, role)` reads a role's fields and applies the same five checks to any role.

```
js
const ROLES = [
  { id: 'CF01', title: 'Data Operations Intern', branches: ['CSE', 'IT'],
    minCgpa: 7.5, gradYears: [2027], maxBacklogs: 1, requiredSkills: ['Python', 'SQL'] },
  // CF02...CF05
];
```

↓

- Pro: adding/removing a role = append/delete one object, zero logic change; one small

+ Write a message...

Opus 4.8 Medium

3. Iteration -- questioning the AI's validation: does Number() let hex / scientific inputs like 2e3 slip through as valid?

Dev work file access

great! what was slipping through as a valid year is now, not a signifier, and the naming tests first for hex and scientific notation on cgpa, year and backlogs, then add the regex guard so only plain decimal input passes. I know it's stricter than the literal spec, note that as a documented assumption. then run tests again

On it — TDD style: tests first, then the guard, plus documenting the assumption.
Edited 3 files, used a tool
Done — 5 new hardening tests added, the regex guard is in, and the stricter-than-spec assumption is documented in `WORKFLOW.md`. Run it:

```
node tests.js
```

Screenshot. Expect: all of Group 2 now green including the 5 new hex/scientific tests — that `2e3` grad year is now correctly rejected. Eligibility groups (1, 4, 5, 6, 7) still red — that's checkpoint 2, next.
(If you want the pure red→green for these specific tests, they'd fail on the old code — I added the guard in the same step for speed. Not worth a separate run unless you want the screenshot.)

Dev work file access

ok validation is solid now, before checkpoint 2, in `evaluateRole` are we lowercasing skills and branches on every single call? is that wasteful or fine at this scale, and would precomputing lowercased sets be cleaner? and should `evaluateRole` defensively handle a malformed role in the config, or just trust it since we already have the group 8 sanity test.

Two good calls to make before I write it, and both point the same way: keep `evaluateRole` clean.

1. Lowercasing per call vs precomputing global sets. At this scale (1 profile × 5 roles × a few skills) the lowercasing cost is microseconds — utterly negligible. Precomputing global lowercased Sets would be a real optimization only at thousands of roles/evaluations, and it has a cost: a *second* copy of the data that has to stay in sync with `ROLES`, plus setup code. For 5 roles that's premature optimization and more moving parts.

The clean middle ground I'll use: don't precompute globally, but *inside* a single `evaluateRole` call, build one lowercased `Set` of the profile's skills and reuse it for the skill checks (instead of re-lowercasing on every comparison). Same for a tiny case-insensitive membership helper for the branch. That's clean and avoids repeated work *within* a call, without duplicating the source data.

Interview line: "I optimized for readability over micro-optimization — precomputing lowercased sets would be premature for five roles and would duplicate the source data. Within a call I build the lowercased skill set once."

2. Defensive handling of a malformed role — no, trust the config. Defensive validation belongs at trust boundaries — untrusted input. The student profile is untrusted (that's why `parseAndValidateProfile` guards it hard). But `ROLES` is *our own compile-time constant*, not external input. Guarding it inside `evaluateRole` would bloat the hot path and duplicate what the Group 8 sanity test already does — and that test ↓ loudly at dev time if someone adds a bad role, which is exactly where you want to catch it.

+ Write a message...

Opus 4.8 Low

4. Refinement, test-first: added failing tests for 2e3 / 0x7EB, then a regex guard so only plain decimals pass. Caught a real gap in the AI's output.

Dev work file access

makes sense, keep evaluateRole lean, build the lowercased skill set once per call, and trust the roles config. now do checkpoint 2, implement evaluateRole with the 5 checks in fixed order no short circuit, case insensitive branch and skill matching, missing skills alpha sorted keeping the role's casing, then evaluateAll, sortResults with eligible first then title then id, and countResults. make everything go green and then I'll run node tests.js

Checkpoint 2 — implementing the engine (lean `evaluateRole`, lowercase-set once per call, trust the config), plus cleaning up the now-stale stub comments.

Edited 5 files, ran a command, used a tool · 2 notes

Now run it yourself to confirm it's green on your machine:

```
node tests.js
```

📸 Screenshot — this is your all-green TDD finish: `Passed: 65 Failed: 0`. That's the full red → green arc documented (empty stub → validation green → engine green).

`logic.js` is done — the whole engine is ~110 lines, and you can walk any function. Next is the UI (`index.html`), and there's a genuine trade-off worth surfacing there before I build it.

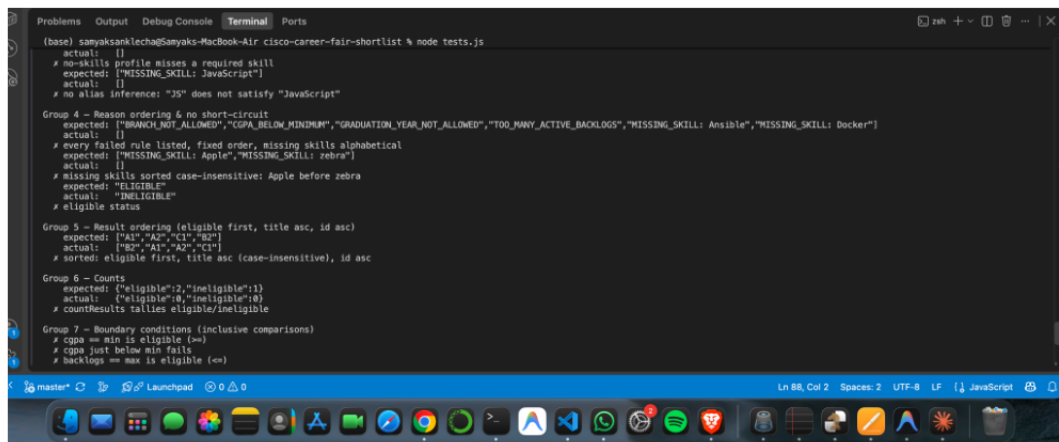
Problems Output Debug Console Terminal Ports

```
(base) samyaksanklecha@sanyaks-MacBook-Air: cisco-career-fair-shortlist % node tests.js
- AC1: counts 3 eligible / 2 ineligible
- AC1: CF04 now Eligible
- AC1: eligible order by title = CF01, CF04, CF02
- no-skills profile misses a required skill
- no alias inference! "JS" does not satisfy "JavaScript"
- every failed rule listed, fixed order, missing skills alphabetical
- missing skills sorted case-insensitive: Apple before zebra
- eligible status
- sorted: eligible first, title asc (case-insensitive), id asc
- countResults tallies eligible/ineligible
- cpga == min is eligible (==)
- cpga just below min fails
- backlogs == max is eligible (==)
- backlogs over max fails
• (base) samyaksanklecha@sanyaks-MacBook-Air: cisco-career-fair-shortlist % node tests.js

Group 1 - Acceptance criteria (from the spec)
Group 2 - Field validation (every input)
Group 3 - Input normalization
Group 4 - Reason ordering & no short-circuit
Group 5 - Result ordering (eligible first, title asc, id asc)
Group 6 - Counts
Group 7 - Boundary conditions (inclusive comparisons)
Group 8 - Data-driven roles config sanity

Passed: 65 Failed: 0
All tests passed
% (base) samyaksanklecha@sanyaks-MacBook-Air: cisco-career-fair-shortlist %
```

5. Test-driven progress: red -> partially green after implementing validation (Failed count dropping).



```
(base) sanyaksanklecha@Sanyaks-MacBook-Air cisco-career-fair-shortlist % node tests.js
actual: {}
x no-skills profile misses a required skill
expected: {MISSING_SKILL: JavaScript}
actual: {}
x no alias inference: "JS" does not satisfy "JavaScript"

Group 4 - Reason ordering & no short-circuit
expected: {BRANCH_NOT_ALLOWED, "CGPA_BELOW_MINIMUM", "GRADUATION_YEAR_NOT_ALLOWED", "TOO_MANY_ACTIVE_BACKLOGS", "MISSING_SKILL: Ansible", "MISSING_SKILL: Docker"}
actual: {}
x every failed rule listed, fixed order, missing skills alphabetical
expected: {MISSING_SKILL: Apple, "MISSING_SKILL: zebra"}
actual: {}
x missing skills sorted case-insensitive: Apple before zebra
expected: "ELIGIBLE"
actual: "INELIGIBLE"
x eligible status

Group 5 - Result ordering (eligible first, title asc, id asc)
expected: {A1: "A2", "C1", "B2"}
actual: {B2: "A1", "A2", "C1"}
x sorted: eligible first, title asc (case-insensitive), id asc

Group 6 - Counts
expected: {eligible: 2, ineligible: 1}
actual: {eligible: 0, ineligible: 0}
x count results tallies eligible/ineligible

Group 7 - Boundary conditions (inclusive comparisons)
x cgpa == min is eligible (==)
x cgpa just below min fails
x backlogs == max is eligible (==)
```

wait, before going further I want to question the validation. why did you use `Number()` instead of `parseInt` or `parseFloat` for `cgpa`, `year` and `backlogs`? and does `Number()` quietly accept weird stuff like hex (`0x7EB`) or scientific notation (`2e3`) that a real user would never type but could slip through as valid? if it does, I want to know whether we should block it or if that's over engineering for this scope

Good question, and there's a real trap here — this isn't a non-issue.

Why `Number()` and not `parseInt`/`parseFloat`: those two are *lenient* — they read as far as they can and silently ignore trailing garbage. That would create false-accepts:

- `parseInt('2027.5')` → `2027` — it would wrongly accept "2027.5" as a valid whole year. That's an actual bug.
- `parseFloat('8.1abc')` → `8.1` — accepts junk.

`Number()` is strict about the *whole* string: `Number('8.1abc')` → `NaN`, `Number('2027.5')` → `2027.5` (then `Number.isInteger` rejects it). So `Number()` + `isFinite`/`isInteger` is the correct choice, and it's a genuine bug avoided. Good instinct to check.

But you caught a real hole: yes, `Number()` accepts non-decimal notation a human would never type:

- `Number('2e3')` → `2000` (would pass as a valid grad year)
- `Number('0x7EB')` → `2027` (hex, would pass)
- `Number('1e1')` → `10` (would pass as CGPA)

So `'2e3'` as graduation year currently slips through as valid. The tests don't catch it because we never test those inputs.

Block it or over-engineering? My honest read: I'd tighten it — it's nearly free and closes a real hole. A one-line regex guard per field (digits-onl ↓ the integers, decimal for CGPA) before `Number()` rejects hex/scientific/signs cleanly. It's *not* over-engineering — it's just making "whole

+ Write a message...

Add folder Manual Opus 4.8 Low

6. TDD complete: all 65 tests green after the eligibility engine was built.

yea all 65 test cases passed and now i think we have checked and discussed wtv we had to for functionality purpose including the trade offs , now i want you to build the index.html file and it should be simple single page with good ui and wired to logic.js file and profile form having 5 roles table with appropriate buttons and results lists , also let me know any trade offs or risk we need to decide on before you start , otherwise go ahead

Thought process

Two genuine ones to flag (I'll apply my recommendation and build, since you said go ahead — course-correct if you disagree):

1. **Risk** — XSS via `innerHTML` . The branch/skills come from user input. If I render results by concatenating HTML strings, a value like `<img onerror=...>` could inject. Low real risk here (offline, single-user — you'd only attack yourself), but the correct habit is to render via `textContent` /DOM methods, not `innerHTML` . **Recommendation: use `textContent`** . Cheap, correct, and a clean thing to say in the room.
2. **Decision** — generate the roles table from `ROLES` , not hardcode it in HTML. Since we made roles data-driven, the reference table should be *built from the data* . **Recommendation: generate it** . Payoff: your live-mod "add a role" edits `ROLES` and the table updates for free — no HTML change.

Both applied. Building `index.html` now.

Creating a file



Creating a file... · 1m 1s

+ Write a message...

